Experiment No. ____

Study and Verification of NOT, OR and EXOR Gates

Aim

To study the NOT, OR and EXOR logic gates using ICs 7404, 7432 and 7486, and verify their truth tables.

Objective

- To understand the logic function and Boolean expression of each gate.
- To identify the pin configuration of the ICs and wire each gate on a breadboard.
- To verify the truth table of each gate practically.

Apparatus / Equipment Required

S.No.	Component	Specification	Quantity
1	IC 7404	Hex Inverter (NOT Gate)	1
2	IC 7432	Quad 2-input OR Gate	1
3	IC 7486	Quad 2-input EXOR Gate	1
4	IC Trainer Kit / Breadboard	-	1
5	Connecting Wires, Power Supply	+5 V DC	As reqd.
6	LEDs / Logic Indicators	-	As reqd.

Diagram

(Space for Gate Symbols and Pin Diagrams)

Theory

Logic gates are the basic building blocks of digital circuits, and each gate performs a Boolean operation on its binary inputs.

NOT gate (IC 7404): a single-input gate whose output is the complement of the input, Y = A'. The IC contains six inverters; input pins are 1, 3, 5, 9, 11, 13.

OR gate (IC 7432): the output is HIGH if at least one input is HIGH, Y = A + B. The IC contains four 2-input OR gates (inputs 1, 2; output 3 for the first gate).

EXOR gate (IC 7486): the output is HIGH only when the two inputs are different, Y = A ⊕ B = A'B + AB'. The IC contains four 2-input EXOR gates (inputs 1, 2; output 3 for the first gate).

For all three ICs, Vcc is pin 14 and GND is pin 7.

Truth Table:

A	B	NOT (A')	OR (A+B)	EXOR (A⊕B)
0	0	1	0	0
0	1	1	1	1
1	0	0	1	1
1	1	0	1	0

K-Map and Boolean Reduction

OR: Y = A + B

A \ B	0	1
0	0	**1**
1	**1**	**1**

EXOR: Y = A⊕B

A \ B	0	1
0	0	**1**
1	**1**	0

OR: looping row A = 1 gives A, and looping column B = 1 gives B, so Y = A + B.

EXOR: the two 1-cells (01 and 10) are diagonal and cannot be grouped, so the expression stays Y = A'B + AB'. NOT: the single-variable map gives Y = A' directly.

Procedure

- Insert the IC on the breadboard/trainer kit and connect Vcc (pin 14) to +5 V and GND (pin 7) to ground.
- NOT gate: connect input A to pin 1 of IC 7404 and the output (pin 2) to an LED. Apply A = 0 and A = 1.
- OR gate: connect inputs A and B to pins 1 and 2 of IC 7432 and the output (pin 3) to an LED.
- EXOR gate: connect inputs A and B to pins 1 and 2 of IC 7486 and the output (pin 3) to an LED.
- For the OR and EXOR gates, apply all four combinations of A and B using logic switches, and note the output each time.

Observation Table

NOT Gate:

S.No.	Input A	Output Y = A'
1	0	
2	1	

OR Gate:

S.No.	A	B	Output Y = A+B
1	0	0	
2	0	1	
3	1	0	
4	1	1	

EXOR Gate:

S.No.	A	B	Output Y = A⊕B
1	0	0	
2	0	1	
3	1	0	
4	1	1	

Result and Analysis

The NOT, OR and EXOR gates were studied using ICs 7404, 7432 and 7486, and the observed outputs matched their truth tables.

Conclusion: _____

Applications

- NOT: signal inversion and complement generation; OR: alarm and combining logic; EXOR: adders, parity generators/checkers and comparators.

References

1. M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 6th ed., Pearson Education, 2018.
2. Thomas L. Floyd, Digital Fundamentals, 11th ed., Pearson Education, 2015.
3. Texas Instruments, SN7404, SN7432 and SN7486 Datasheets.

Experiment No. _____

Construction of Half Adder and Half Subtractor using Logic Gates

Aim

To design, construct, and verify the truth tables of a Half Adder and a Half Subtractor circuit using the logic gate ICs 7404 (NOT), 7408 (AND), and 7486 (XOR).

Objective

- To understand the basic combinational logic circuits used for binary addition and subtraction.
- To study the functioning of NOT, AND, and XOR gates and their combined use in arithmetic circuits.
- To practically implement Half Adder and Half Subtractor circuits on a breadboard/IC trainer kit.
- To verify the theoretical truth table against the practical output obtained.

Apparatus / Equipment Required

S.No.	Component	Specification	Quantity
1	IC 7404 (NOT Gate)	Hex Inverter	1
2	IC 7408 (AND Gate)	Quad 2-input AND Gate	1
3	IC 7486 (XOR Gate)	Quad 2-input XOR Gate	1
4	IC Trainer Kit / Breadboard	-	1
5	Connecting Wires	-	As required
6	Power Supply	+5V DC	1
7	LEDs / Logic Indicators	-	As required

Diagram

(Space for Circuit Diagram)

Theory

A Half Adder is a combinational circuit that performs the addition of two single-bit binary numbers, A and B, producing two outputs: Sum (S) and Carry (C). The Sum output is obtained using an XOR gate (IC 7486), since the Sum is 1 only when the two inputs differ. The Carry output is obtained using an AND gate (IC 7408), since a carry is generated only when both inputs are 1. The logical expressions are: $S = A \oplus B$ and $C = A \cdot B$.

A Half Subtractor is a combinational circuit that performs the subtraction of two single-bit binary numbers, A and B (Difference = A − B), producing two outputs: Difference (D) and Borrow (Bo). The Difference output is obtained using an XOR gate, similar to the Sum in a Half Adder, since $D = A \oplus B$. The Borrow output is generated using a NOT gate (IC 7404) and an AND gate, since a borrow is required only when A is 0 and B is 1. The logical expression is: $Bo = A' \cdot B$, where A' is the complement of A obtained from the NOT gate.

Both circuits are termed 'half' because they do not account for a carry-in or borrow-in from a previous stage, unlike a Full Adder or Full Subtractor. These circuits form the fundamental building blocks of arithmetic logic units (ALUs) used in digital computing systems.

Procedure

- Check all the ICs (7404, 7408, 7486) for their proper functioning using a multimeter or IC tester before use.
- Note the pin diagram of each IC from the datasheet and identify the Vcc and GND pins.
- Insert the ICs onto the breadboard or IC trainer kit, ensuring correct orientation (notch facing the correct direction).
- Connect Vcc (+5V) and GND to the respective pins of all ICs.
- For the Half Adder: connect inputs A and B to one XOR gate (7486) to obtain the Sum output, and to one AND gate (7408) to obtain the Carry output.
- For the Half Subtractor: connect inputs A and B to one XOR gate (7486) to obtain the Difference output. Pass input A through a NOT gate (7404) and feed its output along with B into an AND gate (7408) to obtain the Borrow output.
- Connect the outputs (S, C, D, Bo) to LEDs or logic indicators to observe the output states.
- Apply all four possible combinations of inputs A and B (00, 01, 10, 11) using switches or logic input pins.
- Record the corresponding output readings for each input combination in the observation table.
- Verify the recorded outputs against the theoretical truth table.

Observation Table

Half Adder:

S.No.	A	B	Sum (S)	Carry (C)

Half Subtractor:

S.No.	A	B	Difference (D)	Borrow (Bo)

Result and Analysis

The Half Adder and Half Subtractor circuits were successfully constructed using IC 7404, IC 7408, and IC 7486, and their outputs were verified against the standard truth tables.

Conclusion: _____

_____

_____

Experiment No. _____

Construction of Full Adder and Full Subtractor using Logic Gates

Aim

To design, construct, and verify the truth tables of a Full Adder and a Full Subtractor circuit using the logic gate ICs 7404 (NOT), 7408 (AND), 7486 (XOR), and 7432 (OR).

Objective

- To understand how a Full Adder and Full Subtractor extend the Half Adder and Half Subtractor by accounting for carry-in and borrow-in from a previous stage.
- To study the combined use of NOT, AND, OR, and XOR gates in building multi-bit arithmetic circuits.
- To practically construct Full Adder and Full Subtractor circuits on a breadboard/IC trainer kit.
- To verify the practically observed outputs against the theoretical truth table.

Apparatus / Equipment Required

S.No.	Component	Specification	Quantity
1	IC 7404 (NOT Gate)	Hex Inverter	1
2	IC 7408 (AND Gate)	Quad 2-input AND Gate	1
3	IC 7486 (XOR Gate)	Quad 2-input XOR Gate	1
4	IC 7432 (OR Gate)	Quad 2-input OR Gate	1
5	IC Trainer Kit / Breadboard	-	1
6	Connecting Wires	-	As required
7	Power Supply	+5V DC	1
8	LEDs / Logic Indicators	-	As required

Diagram

(Space for Circuit Diagram)

Theory

A Full Adder is a combinational circuit that adds three single-bit binary inputs: A, B, and a Carry-in (Cin) from the previous stage, producing two outputs: Sum (S) and Carry-out (Cout). It can be built using two Half Adder circuits combined with an OR gate. The first XOR gate produces A ⊕ B, which is then XORed with Cin to give the Sum: S = A ⊕ B ⊕ Cin. The Carry-out is generated whenever at least two of the three inputs are 1, given by the expression: Cout = (A · B) + (Cin · (A ⊕ B)), implemented using two AND gates and one OR gate.

A Full Subtractor is a combinational circuit that subtracts a Borrow-in (Bin) along with input B from input A, producing two outputs: Difference (D) and Borrow-out (Bout). Similar to the Full Adder, it can be built from two Half Subtractor stages combined with an OR gate. The Difference output is given by D = A ⊕ B ⊕ Bin, obtained by cascading two XOR gates. The Borrow-out is generated when A alone is insufficient to perform the subtraction, given by the expression: Bout = (A′ · B) + (B · Bin) + (Bin · A′), implemented using NOT, AND, and OR gates.

These circuits are essential building blocks of Arithmetic Logic Units (ALUs), since cascading multiple Full Adder or Full Subtractor stages (with Cout/Bout of one stage feeding Cin/Bin of the next) allows binary addition and subtraction of multi-bit numbers, forming the basis of ripple-carry adders and subtractors used in digital processors.

Procedure

- Check all the ICs (7404, 7408, 7486, 7432) for proper functioning using a multimeter or IC tester before use.
- Refer to the pin diagram of each IC and identify the Vcc and GND pins.
- Insert the ICs onto the breadboard or IC trainer kit with correct orientation (notch facing the correct direction).
- Connect Vcc (+5V) and GND to the respective pins of all ICs.
- For the Full Adder: feed A and B into an XOR gate, then XOR that output with Cin to obtain Sum. Use two AND gates (for $A \cdot B$ and $Cin \cdot (A \oplus B)$) and feed both into an OR gate to obtain Cout.
- For the Full Subtractor: feed A and B into an XOR gate, then XOR that output with Bin to obtain Difference. Build the Borrow-out logic using the NOT, AND, and OR gates as per the derived expression.
- Connect the outputs (S, Cout, D, Bout) to LEDs or logic indicators to observe the output states.
- Apply all eight possible combinations of the three inputs (A, B, Cin for adder / A, B, Bin for subtractor) using switches or logic input pins.
- Record the corresponding output readings for each input combination in the observation table.
- Verify the recorded outputs against the theoretical truth table for both circuits.

Observation Table

Full Adder:

S.No.	A	B	Cin	Sum (S)	Carry (Cout)

Full Subtractor:

S.No.	A	B	Bin	Difference (D)	Borrow (Bout)

Result and Analysis

The Full Adder and Full Subtractor circuits were successfully constructed using IC 7404, IC 7408, IC 7486, and IC 7432, and their outputs were verified against the standard truth tables.

Conclusion: _____

_____

_____

Experiment No. _____

Construction and Control of a Common Cathode 7-Segment Display using IC 7447

Aim

To construct and control a Common Cathode (CC) 7-Segment Display using IC 7447 (BCD to 7-Segment Decoder/Driver) along with IC 7404 (NOT Gate), and to display the decimal digits 0-9 by applying the corresponding BCD input.

Objective

- To understand the working of the BCD to 7-Segment Decoder/Driver IC 7447.
- To understand the difference between a Common Anode (CA) and a Common Cathode (CC) 7-segment display, and why an inverter stage is required to interface 7447 with a CC display.
- To practically construct the interfacing circuit and verify the displayed digit for each BCD input combination.
- To tabulate and verify the observed segment outputs against the expected 7-segment patterns for digits 0-9.

Apparatus / Equipment Required

S.No.	Component	Specification	Quantity
1	IC 7447	BCD to 7-Segment Decoder/Driver	1
2	IC 7404 (NOT Gate)	Hex Inverter	1
3	7-Segment Display	Common Cathode (CC)	1
4	IC Trainer Kit / Breadboard	-	1
5	Current Limiting Resistors	220Ω - 330Ω	7
6	Connecting Wires	-	As required
7	Power Supply	+5V DC	1

Diagram

(Space for Circuit Diagram)

Theory

A 7-segment display consists of seven individual LED segments, labelled a to g, arranged to form digits when specific segments are illuminated. In a Common Cathode (CC) display, all the cathodes of the seven LEDs are internally connected together to ground, and each segment lights up when its corresponding anode is driven HIGH (logic 1).

IC 7447 is a BCD to 7-Segment Decoder/Driver that accepts a 4-bit BCD input (D C B A) and converts it into the appropriate 7-segment code. However, IC 7447 is designed with active-LOW outputs, meaning each segment output goes LOW to turn that segment ON. This makes it directly suitable for driving a Common Anode (CA) display, but not a Common Cathode (CC) display, which requires active-HIGH signals to turn a segment ON.

To interface IC 7447 with a CC display, each of the seven outputs (a to g) of the 7447 is passed through a NOT gate (IC 7404). This inverts the active-LOW output of the 7447 into an active-HIGH signal, which correctly drives the corresponding segment of the CC display. Since IC 7404 contains six inverters and seven segment lines need inversion, two ICs 7404 are typically required, or one segment line is left to share/reuse a gate configuration as per availability.

The BCD input is applied at pins D, C, B, and A of the 7447. For each of the ten valid BCD combinations (0000 to 1001), the IC decodes the corresponding digit (0-9) and, after inversion through the NOT gates, the correct combination of segments lights up on the CC display to represent that digit. Input combinations beyond 1001 (i.e., 1010 to 1111) are invalid BCD codes and produce non-standard or blank segment patterns.

Procedure

- Check IC 7447, IC 7404, and the CC 7-segment display for proper functioning before use.
- Identify the pin configuration of IC 7447 (BCD inputs D, C, B, A; segment outputs a-g; Vcc and GND) and IC 7404 from their respective pin diagrams.
- Insert the ICs onto the breadboard/IC trainer kit with correct orientation (notch facing the correct direction).
- Connect Vcc (+5V) and GND to the respective pins of both ICs and the display.
- Connect each of the seven segment outputs (a-g) of IC 7447 to the input of a separate NOT gate in IC 7404.
- Connect the output of each NOT gate to the corresponding segment pin (a-g) of the CC 7-segment display, through a current limiting resistor (220Ω-330Ω) in series with each segment.
- Connect the common cathode pin of the display to ground.
- Apply the BCD input combination 0000 (D=0, C=0, B=0, A=0) at the input pins of IC 7447 using switches or logic input pins.
- Observe the digit displayed on the 7-segment display and note which segments are glowing.
- Repeat the above step for all BCD combinations from 0000 to 1001 (decimal 0 to 9), recording the observations each time.
- Verify the displayed digit against the expected output for each BCD input.

Observation Table

S.No.	D	C	B	A	Decimal	Segments Glowing (a-g)

Result and Analysis

The Common Cathode 7-segment display was successfully interfaced with IC 7447 using IC 7404 as an inverting stage, and the decimal digits 0-9 were correctly displayed for their respective BCD input combinations.

Conclusion: _____

_____

_____

Experiment No. _____

Binary to Gray Code and Gray to Binary Code Conversion

Aim

To design, construct, and verify the truth tables of a 4-bit Binary to Gray Code Converter and a 4-bit Gray to Binary Code Converter using IC 7486 (XOR Gate).

Objective

- To understand the concept of Gray code and its advantage over straight binary code, particularly that only one bit changes between successive values.
- To derive the logical expressions for converting a 4-bit binary number into its equivalent Gray code, and vice versa.
- To practically construct both converter circuits using XOR gates on a breadboard/IC trainer kit.
- To verify the practically observed outputs against the theoretical truth table for all 16 input combinations.

Apparatus / Equipment Required

S.No.	Component	Specification	Quantity
1	IC 7486 (XOR Gate)	Quad 2-input XOR Gate	2
2	IC Trainer Kit / Breadboard	-	1
3	Connecting Wires	-	As required
4	Power Supply	+5V DC	1
5	LEDs / Logic Indicators	-	As required

Diagram

(Space for Circuit Diagram)

Theory

Gray code is a binary numeral system in which two successive values differ in only one bit. This property makes it useful in applications such as rotary encoders, Karnaugh maps, and error minimization in digital communication, where straight binary code could cause multiple bits to change simultaneously and lead to transient errors.

Binary to Gray Code Conversion: Let the 4-bit binary number be represented as B3 B2 B1 B0, and the corresponding Gray code as G3 G2 G1 G0. The most significant bit of the Gray code is the same as the most significant bit of the binary number, i.e., G3 = B3. Each subsequent Gray code bit is obtained by XORing the corresponding binary bit with the next higher-order binary bit: G2 = B3 ⊕ B2, G1 = B2 ⊕ B1, and G0 = B1 ⊕ B0. This conversion can be implemented using three XOR gates.

Gray to Binary Code Conversion: This is the reverse process, where the Gray code G3 G2 G1 G0 is converted back to binary B3 B2 B1 B0. The most significant bit remains unchanged, i.e., B3 = G3. However, unlike the binary-to-gray conversion, each binary bit here depends on the previously computed binary bit rather than the original Gray bit alone, since the conversion is cumulative: B2 = B3 ⊕ G2, B1 = B2 ⊕ G1, and B0 = B1 ⊕ G0. This creates a cascaded/rippling XOR structure, where the output of one XOR gate feeds into the next.

Both converters are examples of combinational code converter circuits, and their simplicity using only XOR gates makes them efficient and widely used building blocks in digital systems that interface with Gray-code-based devices such as absolute rotary encoders.

Procedure

- Check the IC 7486 for proper functioning using a multimeter or IC tester before use.
- Note the pin diagram of IC 7486 and identify the Vcc and GND pins.
- Insert the IC(s) onto the breadboard/IC trainer kit with correct orientation (notch facing the correct direction).
- Connect Vcc (+5V) and GND to the respective pins of the IC(s).
- For the Binary to Gray converter: connect G3 directly to B3. Connect B3 and B2 to an XOR gate to obtain G2. Connect B2 and B1 to another XOR gate to obtain G1. Connect B1 and B0 to a third XOR gate to obtain G0.
- For the Gray to Binary converter: connect B3 directly to G3. Feed B3 and G2 into an XOR gate to obtain B2. Feed the resulting B2 and G1 into another XOR gate to obtain B1. Feed the resulting B1 and G0 into a third XOR gate to obtain B0.
- Connect the outputs to LEDs or logic indicators to observe the output states.
- Apply all sixteen possible 4-bit input combinations (0000 to 1111) at the respective inputs using switches or logic input pins, one converter circuit at a time.
- Record the corresponding output readings for each input combination in the observation table.
- Verify the recorded outputs against the theoretical truth table for both converters.

Observation Table

Binary to Gray Code Converter:

S.No.	B3	B2	B1	B0	G3	G2	G1	G0

Gray to Binary Code Converter:

S.No.	G3	G2	G1	G0	B3	B2	B1	B0

S.No.	G3	G2	G1	G0	B3	B2	B1	B0

Result and Analysis

The Binary to Gray Code Converter and Gray to Binary Code Converter circuits were successfully constructed using IC 7486, and their outputs were verified against the standard truth tables for all 16 input combinations.

Conclusion: _____

_____

_____

Applications

- Used in rotary and shaft encoders, where Gray code prevents ambiguous readings caused by multiple bits changing at once.
- Employed in Karnaugh map construction and minimization of digital logic circuits.
- Used in error correction and control systems to minimize transient glitches during signal transitions.
- Applied in analog-to-digital converters (ADCs) and digital communication systems for reliable data transfer.

References

1. M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 6th ed., Pearson Education, 2018.

2. R. P. Jain, Modern Digital Electronics, 4th ed., McGraw Hill Education, 2010.

3. Thomas L. Floyd, Digital Fundamentals, 11th ed., Pearson Education, 2015.

4. Texas Instruments, SN7486 Quadruple 2-Input XOR Gates Datasheet, Texas Instruments Incorporated.

Experiment No. _____

Construction of SR Latch, SR Flip-Flop, and JK Flip-Flop using NAND and AND Gates

Aim

To design, construct, and verify the truth tables of an SR Latch, a clocked SR Flip-Flop, and a JK Flip-Flop using IC 7400 (NAND Gate) and IC 7408 (AND Gate).

Objective

- To understand the concept of sequential circuits and how latches and flip-flops function as basic memory elements.
- To distinguish between a level-sensitive (unclocked) SR Latch and an edge/clock-controlled SR Flip-Flop.
- To derive and minimize the Boolean expressions governing SR and JK flip-flop behaviour using K-Maps.
- To study how the JK flip-flop overcomes the invalid/race-around state present in the SR latch and flip-flop.
- To practically construct all three circuits using NAND and AND gates and verify their truth tables.

Apparatus / Equipment Required

S.No.	Component	Specification	Quantity
1	IC 7400 (NAND Gate)	Quad 2-input NAND Gate	2
2	IC 7408 (AND Gate)	Quad 2-input AND Gate	1
3	IC Trainer Kit / Breadboard	-	1
4	Connecting Wires	-	As required
5	Power Supply	+5V DC	1
6	LEDs / Logic Indicators	-	As required

Diagram

(Space for Circuit Diagram)

Theory

A flip-flop is a bistable sequential circuit that can store one bit of binary information and change its output state only in response to a triggering input (such as a clock edge or gate enable), making it a fundamental building block of registers, counters, and memory units. A latch, in contrast, is level-sensitive: it responds to its inputs continuously, as long as it is enabled, rather than only at a clock transition.

SR Latch

The SR (Set-Reset) Latch is the most basic memory element, constructed using two cross-coupled NAND gates (IC 7400), where the output of each gate is fed back as an input to the other. Since it is built from NAND gates, the inputs are active-LOW, commonly written as S' and R'. When S' = 0 and R' = 1, the latch is Set (Q = 1). When S' = 1 and R' = 0, the latch is Reset (Q = 0). When both S' and R' are HIGH (1), the latch holds its previous output state (memory). When both S' and R' are simultaneously LOW (0), an invalid condition occurs, since both Q and Q' are forced HIGH, violating their complementary relationship, and the resulting state becomes unpredictable once both inputs return HIGH together.

SR Latch Truth Table (NAND-based, active-LOW inputs):

S'	R'	Q	Q'	State
0	0	1	1	Invalid (Not Allowed)
0	1	1	0	Set
1	0	0	1	Reset
1	1	Qn (no change)	Qn' (no change)	Hold / Memory State

The SR Latch has no clock input, meaning its output can change at any time its inputs change, making it unsuitable for use in synchronous systems where predictable timing is required. This limitation is addressed by adding clock-controlled input gating, which leads to the clocked SR Flip-Flop described next.

SR Flip-Flop

The SR (Set-Reset) Flip-Flop is the simplest sequential circuit, built using two cross-coupled NAND gates (IC 7400) along with a clock-enabling AND gate pair (IC 7408) to form the clocked version. The Set (S) input, when HIGH, forces the output Q to 1, and the Reset (R) input, when HIGH, forces Q to 0. When both S and R are LOW, the flip-flop holds its previous state (memory). However, when both S and R are simultaneously HIGH, the circuit enters an invalid/undefined state, since both outputs try to go LOW at once, violating the complementary nature of Q and Q'. This is the primary limitation of the SR flip-flop.

SR Flip-Flop Truth Table:

S	R	Qn+1	State
0	0	Qn (no change)	Hold / Memory State
0	1	0	Reset
1	0	1	Set
1	1	Invalid	Not Allowed (Race condition)

The characteristic equation of the SR flip-flop, derived and minimized using a K-Map with S, R, and Qn as variables (with the S=R=1 condition treated as invalid/don't-care), is: $Qn+1 = S + R' \cdot Qn$, valid only when $S \cdot R = 0$.

JK Flip-Flop

The JK Flip-Flop is an improved version of the SR flip-flop that eliminates the invalid state by feeding the outputs Q and Q' back into the input NAND gates along with the J and K inputs, respectively. This feedback ensures that when both J and K are HIGH, instead of producing an invalid state, the output toggles (switches to its complement) on each clock pulse.

JK Flip-Flop Truth Table:

J	K	Qn+1	State
0	0	Qn (no change)	Hold / Memory State
0	1	0	Reset
1	0	1	Set
1	1	Qn' (Toggle)	Toggle

Excitation Table and K-Map Minimization: The excitation table below lists the required J and K values to achieve a given state transition from Qn to Qn+1, and is used to derive the simplified logic (each cell of the corresponding K-Map contains an X for 'don't care', allowing maximum simplification):

Qn	Qn+1	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Using the K-Map for Qn+1 in terms of J, K, and Qn, the minimized characteristic equation of the JK flip-flop is obtained as: $Qn+1 = J \cdot Qn' + K' \cdot Qn$. This single equation describes all four modes of operation (Hold, Reset, Set, Toggle) and forms the basis for implementing the JK flip-flop using NAND gates, where J and K are combined with the clock and the feedback outputs Q' and Q respectively before being applied to the input latch stage.

Procedure

- Check IC 7400 and IC 7408 for proper functioning using a multimeter or IC tester before use.
- Note the pin diagrams of both ICs and identify the Vcc and GND pins.
- Insert the ICs onto the breadboard/IC trainer kit with correct orientation (notch facing the correct direction).
- Connect Vcc (+5V) and GND to the respective pins of all ICs.

- For the SR Latch: cross-couple two NAND gates directly (output of each gate fed back to an input of the other), with S' and R' applied to the remaining two inputs directly, without any clock gating.
- Apply all four input combinations of S' and R' to the SR Latch and observe the outputs Q and Q' directly, without any clock pulse involved.
- For the SR Flip-Flop: cross-couple two NAND gates (output of each fed back to an input of the other) to form the basic latch. Use two additional NAND gates as clocked input gates, feeding S and Clock into one, and R and Clock into the other, before connecting to the latch.
- For the JK Flip-Flop: extend the clocked SR latch by feeding the Q output back (ANDed with K and Clock) into the R-side input gate, and the Q' output back (ANDed with J and Clock) into the S-side input gate, using the AND gates (7408) for the feedback combination.
- Connect the Q and Q' outputs to LEDs or logic indicators to observe the output states.
- For the SR Flip-Flop, apply all input combinations of S and R (avoiding S=R=1 or specifically testing it to observe the invalid state) along with clock pulses, and record Q before and after each clock pulse.
- For the JK Flip-Flop, apply all input combinations of J and K, including J=K=1, along with clock pulses, and record Q before and after each clock pulse, specifically verifying the toggle action.
- Verify the recorded outputs against the theoretical truth table for both flip-flops.

Observation Table

SR Latch:

S.No.	S'	R'	Q	Q'	State Observed

SR Flip-Flop:

S.No.	Clock	S	R	Qn (Previous)	Qn+1 (Present)

JK Flip-Flop:

S.No.	Clock	J	K	Qn (Previous)	Qn+1 (Present)

Result and Analysis

The SR Latch, SR Flip-Flop, and JK Flip-Flop circuits were successfully constructed using IC 7400 and IC 7408, and their outputs were verified against the standard truth tables. The invalid state observed in both the SR latch (at S'=R'=0) and the SR flip-flop (at S=R=1) was found to be correctly resolved into a toggle operation in the JK flip-flop at J=K=1.

Conclusion: _____

Applications

- Used as basic memory cells in registers, counters, and RAM cells.
- JK flip-flops are widely used in the design of frequency dividers and binary/ripple counters (toggle mode).
- SR latches are used in switch debouncing circuits, simple control/status latching, and as the core storage element inside SR and D-type flip-flops.
- Flip-flops in general form the foundation of finite state machines (FSMs) used in digital controllers.

References

1. M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 6th ed., Pearson Education, 2018.
2. R. P. Jain, Modern Digital Electronics, 4th ed., McGraw Hill Education, 2010.
3. Thomas L. Floyd, Digital Fundamentals, 11th ed., Pearson Education, 2015.
4. Texas Instruments, SN7400 Quadruple 2-Input NAND Gates Datasheet, Texas Instruments Incorporated.

Experiment No. ____

Monostable and Astable Multivibrators using IC 555 Timer

Aim

To construct the monostable and astable multivibrator circuits using IC 555 and study their output waveforms.

Objective

- To understand the one-shot (monostable) and free-running (astable) modes of the 555 timer.
- To observe the output waveforms on the CRO and measure the pulse width, time period and frequency.

Apparatus / Equipment Required

S.No.	Component	Specification	Quantity
1	IC 555 Timer	8-pin DIP	1
2	Resistors	4.6 kΩ, 10 kΩ (astable); 1 kΩ (monostable)	1 each
3	Capacitors	0.1 µF (timing), 0.01 µF (pin 5)	1 each
4	Function Generator / Push Button	Trigger source (monostable)	1
5	Breadboard, Wires, Power Supply	+5 V DC	As reqd.
6	CRO with Probes	Dual channel	1

Diagram

(Space for Astable and Monostable Circuit Diagrams / Waveforms)

Theory

The 555 timer is an 8-pin IC widely used for generating accurate time delays and oscillations. Internally, it has a voltage divider of three equal 5 kΩ resistors, two comparators, an SR flip-flop and a discharge transistor. The divider sets two reference levels, 2/3 Vcc for the threshold comparator (pin 6) and 1/3 Vcc for the trigger comparator (pin 2). The flip-flop is set or reset when the voltage across the external timing capacitor crosses these levels, and this decides whether the output (pin 3) is HIGH or LOW. The discharge transistor (pin 7) is used to discharge the capacitor. The 0.01 µF capacitor at pin 5 does not affect the timing, and only filters noise from the control voltage.

Monostable Mode

A monostable multivibrator, also called a one-shot, has one stable state and one temporary (quasi-stable) state. In the stable state, the output is LOW and the discharge transistor is ON, which keeps the capacitor discharged. The circuit stays in this state until a trigger is applied.

When a negative-going trigger pulse falls below 1/3 Vcc at pin 2, the trigger comparator sets the flip-flop. The output goes HIGH and the discharge transistor turns OFF, so the capacitor starts charging through R. When the capacitor voltage reaches 2/3 Vcc, the threshold comparator resets the flip-flop. The output returns to LOW and the capacitor is discharged quickly, so the circuit waits for the next trigger.

The width of the output pulse depends only on R and C, and is given by T = 1.1 R C. It does not depend on the trigger pulse width, as long as the trigger is shorter than T and its period is longer than T.

Astable Mode

An astable multivibrator has no stable state, and it keeps switching between HIGH and LOW by itself. No external trigger is required, and it works as a free-running oscillator. Pins 2 and 6 are joined together, so the capacitor voltage is monitored by both comparators.

When the supply is switched ON, the capacitor charges through RA and RB towards Vcc, and the output is HIGH. When the voltage reaches 2/3 Vcc, the flip-flop is reset, the output goes LOW and the discharge transistor turns ON. The capacitor now

discharges through RB only, until its voltage falls to 1/3 Vcc. Then the flip-flop is set, the output goes HIGH again and the capacitor charges once more. This cycle repeats continuously and gives a rectangular wave at the output.

As the capacitor charges through (RA + RB) but discharges through RB only, the HIGH time is longer than the LOW time. The timing is given by:

$T_{ON} = 0.693\,(R_A + R_B)\,C, \quad T_{OFF} = 0.693\,R_B\,C, \quad T = T_{ON} + T_{OFF}, \quad f = 1 / T$

Duty cycle = $(T_{ON} / T) \times 100$

Procedure

Monostable

- Connect pin 1 to GND, pins 8 and 4 to +5 V, and R (1 kΩ) between +5 V and pin 7. Join pins 6 and 7.
- Connect C (0.1 µF) from pin 6 to GND, and 0.01 µF from pin 5 to GND.
- Apply a negative trigger pulse at pin 2, and observe the trigger and output (pin 3) on the CRO. Note the output pulse width.

Astable

- Connect pin 1 to GND, pins 8 and 4 to +5 V. Connect RA (4.6 kΩ) between +5 V and pin 7, and RB (10 kΩ) between pin 7 and pin 6. Short pins 6 and 2.
- Connect C (0.1 µF) from pin 6 to GND, and 0.01 µF from pin 5 to GND.
- Observe the output at pin 3 on the CRO, and note T_ON, T_OFF and the total time period. Calculate f and the duty cycle.

Observation Table

Monostable:

S.No.	Trigger Applied	Output Pulse Width
1	Yes	
2	Yes	
3	Yes	

Astable:

Parameter	Observed Value
ON time (T_ON)	
OFF time (T_OFF)	
Total time period (T)	
Frequency (f)	
Duty cycle	

Result and Analysis

The monostable and astable multivibrators were constructed using IC 555. The monostable circuit gave a single output pulse of fixed width for each trigger, and the astable circuit gave a continuous rectangular wave. The pulse width, time period and frequency were measured from the CRO and recorded in the observation tables.

Conclusion: _____

Applications

- Monostable: time delays, pulse-width stretching, switch debouncing.
- Astable: clock generators, LED flashers, tone and siren generators.

References

1. Ramakant A. Gayakwad, Op-Amps and Linear Integrated Circuits, 4th ed., Pearson Education, 2000.
2. D. Roy Choudhury and Shail B. Jain, Linear Integrated Circuits, 5th ed., New Age International, 2018.
3. Texas Instruments, NE555 Precision Timers Datasheet, Texas Instruments Incorporated.

Experiment No. ____

3-to-8 Line Decoder (IC 74138) and 8:1 Multiplexer (IC 74151)

Aim

To study and verify the truth tables of a 3-to-8 line decoder using IC 74138 and an 8:1 multiplexer using IC 74151.

Objective

- To understand how a decoder activates one unique output line for each binary select combination.
- To understand how a multiplexer routes one of several data inputs to a single output using select lines.
- To verify both ICs practically against their standard truth tables.

Apparatus / Equipment Required

S.No.	Component	Specification	Quantity
1	IC 74138	3-to-8 Line Decoder	1
2	IC 74151	8:1 Multiplexer	1
3	IC Trainer Kit / Breadboard	-	1
4	Connecting Wires, Power Supply	+5 V DC	As reqd.
5	LEDs / Logic Indicators	-	As reqd.

Diagram

(Space for Circuit / Pin Diagram)

Theory

IC 74138 is a 3-to-8 line decoder with select lines S2, S1, S0 and eight active-LOW outputs Y0-Y7. It is enabled only when EN1 = 1 and EN2' = EN3' = 0; otherwise all outputs stay HIGH. Each output corresponds to one minterm of the select inputs, so Yn goes LOW only for its matching S2S1S0 combination. Since each output already represents a single minterm, it cannot be simplified further; a decoder is, in effect, a hardware realization of every 3-variable minterm, which is why decoders are used to directly implement any Boolean function in sum-of-minterms form.

IC 74151 is an 8:1 multiplexer with data inputs D0-D7, select lines A, B, C and an active-LOW enable EN'. When enabled, the output Y equals the data input selected by A, B, C, given by Y = A'B'C'D0 + A'B'CD1 + ... + ABCD7. This expression is a direct Sum-of-Products over the 3-variable K-Map of A, B, C, with each Di acting as the value of that K-Map cell, so a MUX can realize any 3-variable function by tying its Di inputs to 0, 1, or a variable as read off the map, without separate gate-level minimization.

K-Map and Algebraic Reduction (Decoder)

Each decoder output is exactly one cell of the 3-variable K-Map of S2, S1, S0 (cell contents show which output owns it). A single minterm cannot be grouped, so no larger loop exists; instead, the active-LOW output reduces by De Morgan's theorem:

S2 \ S1S0	00	01	11	10
0	Y0	Y1	Y3	Y2
1	Y4	Y5	Y7	Y6

```
Y0' = (S2'S1'S0')'  =  S2 + S1 + S0          (De Morgan)
Y5' = (S2 S1' S0)'  =  S2' + S1 + S0'
```

Use as a function generator: the Full Adder outputs Sum = Σm(1,2,4,7) and Carry = Σm(3,5,6,7) need no K-Map reduction when taken from a decoder, since the active-LOW outputs are simply NANDed:

```
Sum = (Y1 · Y2 · Y4 · Y7)'      Carry = (Y3 · Y5 · Y6 · Y7)'
```

K-Map and Algebraic Reduction (Multiplexer)

To realize F(A,B,C) = Σm(1,2,6,7) with the 8:1 MUX, first minimize on a K-Map (loops: AB = m6,m7; BC' = m2,m6; A'B'C = m1 alone):

A \ BC	00	01	11	10
0	0	**1**	0	**1**
1	0	0	**1**	**1**

```
F = AB + BC' + A'B'C
```

On the MUX, the minimization is not even needed: tie D1 = D2 = D6 = D7 = 1 and D0 = D3 = D4 = D5 = 0 (select lines A B C), and the IC directly produces F = Σm(1,2,6,7).

Procedure

Decoder

- Insert IC 74138, connect Vcc and GND, and wire S0-S2 and EN1, EN2', EN3' to logic switches.
- Connect Y0-Y7 to LEDs. Apply EN1=1, EN2'=0, EN3'=0, then step S2S1S0 through 000 to 111 and note which output goes LOW each time.

Multiplexer

- Insert IC 74151, connect Vcc and GND, and wire A, B, C and EN' to logic switches, with D0-D7 set to distinct known levels.
- Apply EN'=0, then step A B C through 000 to 111 and verify that Y matches the expected Dn each time.

Observation Table

3-to-8 Decoder:

S.No.	EN3'	EN2'	EN1	S2	S1	S0	Active-LOW Output
1							
2							
3							
4							
5							
6							
7							
8							

8:1 Multiplexer:

S.No.	A	B	C	EN'	Output Y
1					
2					
3					
4					
5					
6					
7					
8					

Result and Analysis

The decoder and multiplexer were verified: the decoder activated exactly one output LOW per select combination, and the multiplexer correctly routed each selected data input to the output.

Conclusion: _____

Applications

- Decoder: memory address decoding, instruction decoding, 1-of-8 chip-select, and implementing Boolean functions in minterm form.

- Multiplexer: data routing and selection, parallel-to-serial conversion, and realizing arbitrary logic functions.

References

[1] M. M. Mano and M. D. Ciletti, Digital Design, 6th ed., Pearson, 2018. [2] R. P. Jain, Modern Digital Electronics, 4th ed., McGraw Hill, 2010. [3] Texas Instruments, SN74138 and SN74151 Datasheets.

Experiment No. ____

4-bit Magnitude Comparator using IC 74LS85

Aim

To study the working of a 4-bit magnitude comparator using IC 74LS85 and verify its outputs for different pairs of 4-bit numbers.

Objective

- To understand how two multi-bit binary numbers are compared bit by bit, starting from the most significant bit.
- To derive the Boolean condition for equality and inequality of two numbers using K-Map logic for a single bit pair.
- To verify the IC's A>B, A=B and A<B outputs practically, including the use of its cascading inputs.

Apparatus / Equipment Required

S.No.	Component	Specification	Quantity
1	IC 74LS85	4-bit Magnitude Comparator	1
2	IC Trainer Kit / Breadboard	-	1
3	Connecting Wires, Power Supply	+5 V DC	As reqd.
4	LEDs / Logic Indicators	-	As reqd.

Diagram

(Space for Circuit / Pin Diagram)

Theory

A magnitude comparator compares two binary numbers, A and B, and indicates whether A is greater than, equal to, or less than B. IC 74LS85 compares two 4-bit numbers, A3A2A1A0 and B3B2B1B0, and provides three outputs: A>B, A=B and A<B. It also has three cascading inputs (IA>B, IA=B, IA<B) that let several 74LS85 ICs be chained to compare numbers wider than 4 bits, with the least significant stage's outputs feeding the cascading inputs of the next higher stage.

The comparison is done bit by bit, starting from the most significant bit. The logic is first derived for one pair of bits (Ai, Bi) using K-Maps, and then extended to four bits.

K-Map Reduction (1-bit comparator)

A > B

A \ B	0	1
0	0	0
1	**1**	0

(A>B) = A B'

A < B

A \ B	0	1
0	0	**1**
1	0	0

(A<B) = A' B

A = B

A \ B	0	1
0	**1**	0
1	0	**1**

(A=B) = A'B' + AB

The two 1-cells of A=B are diagonal, so they cannot be grouped and the XNOR form is already minimal. It can be verified algebraically:

```
(A xor B)' = (AB' + A'B)' = (A' + B)(A + B')
           = A'A + A'B' + AB + BB' = AB + A'B'      (since A'A = BB' = 0)
```

Extension to 4 bits

Let Ei = AiBi + Ai'Bi' (bit i equal). Then A = B only when all four bits are equal, and A > B (or A < B) is decided at the first, most significant bit where the numbers differ:

```
(A=B) = E3 E2 E1 E0
(A>B) = A3B3' + E3 A2B2' + E3E2 A1B1' + E3E2E1 A0B0'
(A<B) = A3'B3 + E3 A2'B2 + E3E2 A1'B1 + E3E2E1 A0'B0
```

These are priority (cascaded) conditions across eight input variables, so a single flat K-Map is not practical; the 1-bit K-Map results above are repeated at each bit position.

Procedure

- Insert IC 74LS85, connect Vcc and GND, and wire A3-A0, B3-B0 to logic switches.
- For a single (non-cascaded) IC, set IA>B = 0, IA=B = 1, IA<B = 0 (the standard non-cascading condition).
- Connect the A>B, A=B and A<B outputs to LEDs.
- Apply various 4-bit combinations of A and B (equal, A greater, A smaller) and note which output goes HIGH each time.
- Verify the outputs against the expected comparison result for each input pair.

Observation Table

S.No.	A3A2A1A0	B3B2B1B0	A>B	A=B	A<B
1					
2					
3					
4					
5					
6					
7					
8					

Result and Analysis

The 4-bit magnitude comparator was verified using IC 74LS85. For every tested pair of 4-bit numbers, exactly one of the three outputs (A>B, A=B, A<B) was found HIGH, matching the expected comparison.

Conclusion: _____

Applications

- Arithmetic units, sorting and ranking circuits, and priority/threshold detection (e.g. a digital thermostat).
- Address comparison in memory/cache systems, and wider comparators built by cascading several 74LS85 ICs.

References

[1] M. M. Mano and M. D. Ciletti, Digital Design, 6th ed., Pearson, 2018. [2] T. L. Floyd, Digital Fundamentals, 11th ed., Pearson, 2015. [3] Texas Instruments, SN74LS85 4-Bit Magnitude Comparator Datasheet.

Experiment No. _____

NOT Gate using IC CD4007

Aim

To construct a NOT gate (inverter) using IC CD4007 and verify its truth table.

Objective

- To understand the internal structure of IC CD4007 as an array of NMOS and PMOS transistors.
- To connect one complementary MOS pair of CD4007 as a CMOS inverter and verify its output.

Apparatus / Equipment Required

S.No.	Component	Specification	Quantity
1	IC CD4007	CMOS Dual Complementary Pair + Inverter	1
2	IC Trainer Kit / Breadboard	-	1
3	Connecting Wires, Power Supply	+5 V DC (3-15 V range)	As reqd.
4	LED / Logic Indicator	-	1

Diagram

(Space for Circuit / Pin Diagram)

Theory

IC CD4007 is not a dedicated logic-gate IC; it contains three N-channel and three P-channel enhancement-mode MOSFETs, independently accessible at its pins, which can be wired to form inverters, NAND, NOR or other CMOS gates as required. To build a single NOT gate, one N-channel and one P-channel transistor are connected as a complementary pair: their gates are tied together to form the input A, their drains are tied together to form the output Y, the PMOS source is tied to Vdd, and the NMOS source is tied to ground.

When A is LOW (0), the PMOS conducts and the NMOS is OFF, pulling the output Y to Vdd (logic 1). When A is HIGH (1), the NMOS conducts and the PMOS is OFF, pulling Y to ground (logic 0). Hence Y = A' in both cases.

K-Map and Algebraic Reduction

A	0	1
Y	**1**	0

The only 1-cell is A = 0, so the single loop gives Y = A'. The function is already in its simplest form, and it satisfies the complement identities:

```
Y + A = A' + A = 1        Y . A = A' . A = 0        (Y')' = A
```

Procedure

- Identify the CD4007 pin diagram and locate one complementary MOS pair (e.g., one P-channel and one N-channel transistor sharing accessible gate, drain and source pins).
- Connect Vdd and GND (Vss) to the appropriate supply pins.
- Tie the gates of the chosen P-channel and N-channel transistors together to form input A.
- Tie their drains together to form output Y; connect the PMOS source to Vdd and the NMOS source to GND.
- Connect input A to a logic switch and output Y to an LED or logic indicator.
- Apply A = 0 and A = 1 in turn, and note the resulting output Y each time.

Observation Table

S.No.	Input A	Output Y = A'
1	0	
2	1	

Result and Analysis

A NOT gate was successfully constructed using one complementary MOS pair of IC CD4007. The output was found to be the exact complement of the input in both logic states, verifying correct inverter operation.

Conclusion: _____

Applications

- Basic inverter / logic-level reversal and buffer stages in low-power CMOS circuits.
- Building block for custom NAND/NOR gates, ring oscillators and small logic circuits using the same CD4007 array.

References

[1] T. L. Floyd, Digital Fundamentals, 11th ed., Pearson, 2015. [2] R. P. Jain, Modern Digital Electronics, 4th ed., McGraw Hill, 2010. [3] Texas Instruments, CD4007UB CMOS Dual Complementary Pair Plus Inverter Datasheet.

Experiment No. _____

NAND and NOR Gates using IC CD4007

Aim

To construct a 2-input NAND gate and a 2-input NOR gate using the MOS transistors of IC CD4007 and verify their truth tables.

Objective

- To understand how CMOS logic gates are built from a pull-up (PMOS) and a pull-down (NMOS) network.
- To connect the P-channel and N-channel transistors of CD4007 to form NAND and NOR gates.
- To verify the truth table of each gate practically, and derive the output using K-Map and Boolean algebra.

Apparatus / Equipment Required

S.No.	Component	Specification	Quantity
1	IC CD4007	CMOS Dual Complementary Pair + Inverter	1
2	IC Trainer Kit / Breadboard	-	1
3	Connecting Wires, Power Supply	+5 V DC (3-15 V range)	As reqd.
4	LED / Logic Indicator (with 330 Ω resistor)	-	1

Diagram

(Space for NAND and NOR Circuit Diagrams)

Theory

IC CD4007 contains three N-channel and three P-channel MOSFETs, accessible at separate pins, with the supply on pin 14 (Vdd) and pin 7 (Vss/GND). Like the NOT gate, the NAND and NOR gates are built by wiring these transistors in the CMOS style: a PMOS pull-up network connects the output Y to Vdd, and an NMOS pull-down network connects Y to ground. Exactly one network conducts for any input, so Y is always driven HIGH or LOW. Each input drives the gate of one PMOS and one NMOS transistor.

NAND Gate

The two PMOS transistors are connected in parallel between Vdd and Y, and the two NMOS transistors are connected in series between Y and ground. Y goes LOW only when both NMOS are ON, i.e. when A = B = 1. If either input is 0, one PMOS turns ON and pulls Y HIGH.

NOR Gate

The two PMOS transistors are connected in series between Vdd and Y, and the two NMOS transistors are connected in parallel between Y and ground. Y goes HIGH only when both PMOS are ON, i.e. when A = B = 0. If either input is 1, one NMOS turns ON and pulls Y LOW.

Truth Table:

A	B	NAND Y = (AB)'	NOR Y = (A+B)'
0	0	1	1
0	1	1	0
1	0	1	0
1	1	0	0

K-Map and Boolean Reduction

NAND: Y = (AB)'

A \ B	0	1
0	**1**	**1**
1	**1**	0

NOR: Y = (A+B)'

A \ B	0	1
0	**1**	0
1	0	0

NAND: the 1-cells are 00, 01 and 10. Looping row A = 0 gives A', and looping column B = 0 gives B'. Hence Y = A' + B', which is (AB)' by De Morgan's theorem.

NOR: only cell 00 is 1, so the single loop gives Y = A'B', which is (A + B)' by De Morgan's theorem.

```
NAND:  Y = (AB)'  = A' + B'        NOR:  Y = (A+B)'  = A'B'
```

The conducting networks follow the same algebra: for NAND, the series NMOS conduct for AB and the parallel PMOS conduct for A' + B'; for NOR, the parallel NMOS conduct for A + B and the series PMOS conduct for A'B'.

Procedure

NAND Gate

- Refer to the CD4007 pin diagram in the datasheet. Connect Vdd (pin 14) to +5 V and Vss (pin 7) to GND.
- Take two P-channel transistors, and connect both sources to Vdd and both drains together to the output Y (parallel).
- Take two N-channel transistors in series: connect the drain of the first to Y, its source to the drain of the second, and the source of the second to GND.
- Connect the gates of the first PMOS and first NMOS together as input A, and the gates of the second PMOS and second NMOS together as input B.
- Connect A and B to logic switches and Y to an LED. Apply all four input combinations, and note the output.

NOR Gate

- Take two P-channel transistors in series: connect the source of the first to Vdd, its drain to the source of the second, and the drain of the second to Y.
- Take two N-channel transistors in parallel: connect both drains to Y and both sources to GND.
- Connect the gates as before (first PMOS and NMOS to A, second PMOS and NMOS to B), apply all four input combinations, and note the output.

Note: tie the gates of any unused CD4007 transistors to Vdd or GND, since unconnected CMOS inputs can pick up noise.

Observation Table

NAND Gate:

S.No.	A	B	Output Y (NAND)
1	0	0	
2	0	1	
3	1	0	
4	1	1	

NOR Gate:

S.No.	A	B	Output Y (NOR)
1	0	0	
2	0	1	
3	1	0	
4	1	1	

Result and Analysis

The NAND and NOR gates were constructed using the MOS transistors of IC CD4007. The NAND output was LOW only when both inputs were HIGH, and the NOR output was HIGH only when both inputs were LOW, verifying the truth tables of both gates.

Conclusion: _____

Applications

- NAND and NOR are universal gates, so any logic circuit can be built from either one.
- Basic cells of CMOS logic design, used in low-power digital ICs such as microprocessors and memories.

References

1. Thomas L. Floyd, Digital Fundamentals, 11th ed., Pearson Education, 2015.
2. R. P. Jain, Modern Digital Electronics, 4th ed., McGraw Hill Education, 2010.
3. Neil H. E. Weste and David Money Harris, CMOS VLSI Design: A Circuits and Systems Perspective, 4th ed., Pearson, 2011.
4. Texas Instruments, CD4007UB CMOS Dual Complementary Pair Plus Inverter Datasheet.

Experiment No. _____

Implementation of a Half Adder in Verilog HDL using Xilinx ISE

Aim

To write the Verilog HDL code for a Half Adder using gate-level modeling, and to verify it through syntax check, behavioral simulation with a test fixture, and synthesis (RTL schematic) in Xilinx ISE.

Objective

- To learn the complete HDL design flow in Xilinx ISE: project creation, design entry, syntax check, simulation and synthesis.
- To describe a combinational circuit in Verilog using built-in gate primitives (xor, and).
- To write a Verilog test fixture and verify the design from the simulation waveform.
- To obtain and study the RTL schematic generated after synthesis.

Software Required

S.No.	Software / Tool	Details
1	Xilinx ISE Design Suite	Project Navigator (HDL design entry, synthesis)
2	XST (Xilinx Synthesis Technology)	Synthesis and RTL schematic generation
3	ISim Simulator	Behavioral simulation and waveform viewer
4	Computer	Windows PC with ISE installed

Theory

A Half Adder is a combinational circuit that adds two single-bit inputs, x and y, and produces two outputs: Sum (s) and Carry (c). The Sum is the XOR of the inputs, and the Carry is the AND of the inputs:

$s = x \oplus y, \quad c = x \cdot y$

x	y	Sum (s)	Carry (c)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Verilog HDL is a hardware description language used to describe digital circuits at different levels of abstraction. In gate-level (structural) modeling, the circuit is described using built-in gate primitives such as and, or, xor and not. For these primitives, the first terminal is the output and the remaining terminals are the inputs, so xor(s, x, y) means s = x XOR y.

Xilinx ISE is a design environment for FPGA/CPLD design. Its Project Navigator manages the project, the XST tool synthesizes the Verilog code into a netlist, and the ISim simulator is used to verify the design using a test fixture (testbench), which applies input patterns to the Unit Under Test (UUT) and displays the outputs as a waveform. Since a test fixture only drives the design for simulation, it has no ports of its own and is not synthesized.

Procedure

Capture a screenshot after each step, as marked by the figure numbers, and attach them to the lab file.

A. Creating a New Project

Step 1: Open Project Navigator. Launch Xilinx ISE Design Suite, and select File > New Project. *(Fig. 1)*

Step 2: Enter project details. In the New Project Wizard, type the project name (e.g. half_adder), choose the location, and set Top-Level Source Type as HDL. Click Next. *(Fig. 1)*

Step 3: Set project properties. Select the Family, Device, Package and Speed as specified for the lab, and set Synthesis Tool as XST (VHDL/Verilog), Simulator as ISim (VHDL/Verilog) and Preferred Language as Verilog. Click Next. *(Fig. 2)*

Step 4: Finish. Check the Project Summary and click Finish. The empty project opens in Project Navigator. *(Fig. 3)*

Property	Value
Top-Level Source Type	HDL
Preferred Language	Verilog
Synthesis Tool	XST (VHDL/Verilog)
Simulator	ISim (VHDL/Verilog)
Family / Device / Package / Speed	_____ (as per lab)

B. Creating the Verilog Module

Step 5: Add a new source. Select Project > New Source (or right-click the device in the Hierarchy panel and choose New Source). *(Fig. 4)*

Step 6: Choose source type. Select Verilog Module, type the file name (half_adder), and click Next. *(Fig. 4)*

Step 7: Define the ports. In the Define Module window, enter the port names and directions: x (input), y (input), s (output) and c (output). Click Next and then Finish. *(Fig. 5)*

Step 8: Write the code. The module opens in the editor. Enter the following gate-level code and save the file. *(Fig. 6)*

```
module half_adder(x, y, s, c);
    input x;
    input y;
    output s;
    output c;

    xor (s, x, y);   // Sum = x XOR y
    and (c, x, y);   // Carry = x AND y
endmodule
```

C. Syntax Check

Step 9: Select Implementation view. In the Design panel, set the view to Implementation and select half_adder.v in the Hierarchy.

Step 10: Run Check Syntax. In the Processes panel, expand Synthesize - XST and double-click Check Syntax. A green tick mark and the message “Process Check Syntax completed successfully” in the Console confirm that the code is free of syntax errors. If errors appear, correct them and run again. *(Fig. 7)*

D. Creating the Test Fixture

Step 11: Add a test fixture. Select Project > New Source, choose Verilog Test Fixture, enter the file name (half_adder_tb) and click Next. *(Fig. 8)*

Step 12: Associate the source. Select half_adder as the module to be tested, and click Next and Finish. ISE generates the test fixture with the UUT instantiated. *(Fig. 8)*

Step 13: Apply the stimulus. In the initial block, add the input patterns with a delay of #100 between each, so that all four combinations of x and y are applied. Save the file. *(Fig. 9)*

```
module half_adder_tb;
    // Inputs
    reg x;
    reg y;
    // Outputs
    wire s;
    wire c;

    // Instantiate the Unit Under Test (UUT)
    half_adder uut (.x(x), .y(y), .s(s), .c(c));

    initial begin
        x = 0; y = 0;
        #100;  x = 0; y = 1;
        #100;  x = 1; y = 0;
        #100;  x = 1; y = 1;
        #100;
    end
endmodule
```

E. Behavioral Simulation

Step 14: Switch to Simulation view. In the Design panel, set the view to Simulation, and select half_adder_tb in the Hierarchy. *(Fig. 10)*

Step 15: Run the simulation. In the Processes panel, expand ISim Simulator and double-click Simulate Behavioral Model. The ISim window opens. *(Fig. 10)*

Step 16: Study the waveform. Use Zoom to Full View to see the complete run. Observe x, y, s and c for each 100 ns interval, and note the values in the observation table. *(Fig. 11)*

F. Synthesis and RTL Schematic

Step 17: Return to Implementation view. Set the view back to Implementation and select half_adder.

Step 18: Synthesize the design. Double-click Synthesize - XST in the Processes panel, and wait for the green tick mark showing successful synthesis. *(Fig. 12)*

Step 19: View the RTL schematic. Under Synthesize - XST, double-click View RTL Schematic. In the startup dialog, choose to start with the schematic of the top-level block, and click OK. The block with inputs x, y and outputs s, c is displayed. *(Fig. 13)*

Step 20: Expand the block. Double-click the block to see the internal circuit, in which an XOR gate drives s and an AND gate drives c. *(Fig. 14)*

Optionally, View Technology Schematic can be opened in the same way to see how the design is mapped to the FPGA resources.

Diagram

(Attach RTL Schematic and Simulation Waveform screenshots)

Observation Table

Values observed from the simulation waveform:

S.No.	Time Interval (ns)	x	y	Sum (s)	Carry (c)
1	0 – 100	0	0		
2	100 – 200	0	1		
3	200 – 300	1	0		
4	300 – 400	1	1		

Result and Analysis

The half adder was described in Verilog HDL using gate primitives, and successfully checked for syntax errors, simulated and synthesized in Xilinx ISE. The simulation waveform showed that the Sum (s) is HIGH only when exactly one input is HIGH, and the Carry (c) is HIGH only when both inputs are HIGH, matching the half adder truth table. The RTL schematic showed one XOR gate and one AND gate, as expected from the code.

Conclusion: _____

_____

Applications

- Basic building block of full adders, ripple-carry adders and the ALU of a processor.
- Illustrates the standard HDL design flow (entry, simulation, synthesis) used for FPGA and ASIC designs.

References

1. Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed., Pearson Education, 2003.

2. M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 6th ed., Pearson Education, 2018.

3. Xilinx Inc., ISE In-Depth Tutorial (UG695).

Screenshot Index

Use this table to label the printed screenshots in the lab file:

Fig. No.	Description	Page No.
1	New Project Wizard: project name and location	
2	Project settings (family, device, language)	
3	Project summary / project opened in Project Navigator	
4	New Source Wizard: Verilog Module selected	
5	Define Module: input and output ports	
6	Verilog code of half adder in the editor	
7	Check Syntax completed successfully	
8	New Source Wizard: Verilog Test Fixture, source association	
9	Test fixture code with stimulus	
10	Simulation view with Simulate Behavioral Model	
11	ISim output waveform (x, y, s, c)	
12	Synthesize – XST completed successfully	
13	RTL Schematic: top-level block	
14	RTL Schematic: internal XOR and AND gates	